StreamRL: Scalable, Heterogeneous, and Elastic RL for LLMs with Disaggregated Stream Generation

Yinmin Zhong[1] Zili Zhang[1] Xiaoniu Song[2] Hanpeng Hu[2]
Chao Jin[1] Bingyang Wu[1] Nuo Chen[2] Yukun Chen[2] Yu Zhou[2]
Changyi Wan[2] Hongyu Zhou[2] Yimin Jiang[3] Yibo Zhu[2] Daxin Jiang[2]

[1]*School of Computer Science, Peking University* [2]*StepFun* [3]*Unaffiliated*

arXiv:2504.15930v1 [cs.LG] 22 Apr 2025

Yinmin Zhong1

Abstract

Reinforcement learning (RL) has become the core post-training technique for large language models (LLMs). RL for LLMs involves two stages: generation and training. The LLM first generates samples online, which are then used to derive rewards for training. The conventional view holds that the *colocated architecture*—where the two stages share resources via temporal multiplexing—outperforms the *disaggregated architecture*, in which dedicated resources are assigned to each stage. However, in real-world deployments, we observe that the colocated architecture suffers from *resource coupling*, where the two stages are constrained to use the same resources. This coupling compromises the scalability and cost-efficiency of colocated RL in large scale training. In contrast, the disaggregated architecture allows for flexible resource allocation, supports heterogeneous training setups, and facilitates cross-datacenter deployment.

StreamRL is designed with disaggregation from first principles and fully unlocks its potential by addressing two types of performance bottlenecks in existing disaggregated RL frameworks: *pipeline bubbles*, caused by stage dependencies, and *skewness bubbles*, resulting from long-tail output length distributions. To address pipeline bubbles, StreamRL breaks the traditional stage boundary in synchronous RL algorithms through stream generation, and achieves fully overlapping in asynchronous RL. To address skewness bubbles, StreamRL employs an output-length ranker model to identify long-tail samples and reduces generation time via skewness-aware dispatching and scheduling. Experiments show that StreamRL improves throughput by up to 2.66× compared to existing state-of-the-art systems, and improves cost-effectiveness by up to 1.33× in heterogeneous, cross-datacenter setting.

1 Introduction

Reinforcement learning (RL) has emerged as a new paradigm for training large language models (LLMs), substantially improving their reasoning capabilities and revealing a novel inference-time scaling law [7, 13, 53]. State-of-the-art models such as OpenAI o1 [2] and o3 [4], Claude 3.7 Sonnet [6], and DeepSeek-R1 [13] have all adopted RL to achieve leading performance in tasks such as coding and mathematics.

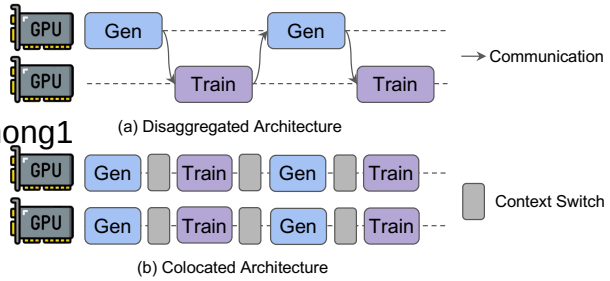

Figure 1. Two representative RL framework architectures.

In contrast to traditional next-token prediction [10, 11, 32, 45, 46] in pre-training, RL enables the LLMs to learn by trial and error from reward signals. While numerous RL algorithms exist, such as PPO [36] and GRPO [37], the typical RL workflow for LLMs involves two main stages in serial: generation and training. In generation stage, the LLM produces samples on a batch of given prompts, followed by the training stage that updates the LLM based on rewards derived from the generated samples.

Due to this two-stage workflow, initial RL training frameworks for LLMs, such as OpenRLHF [19] and NeMo [18], naturally adopted a *disaggregated architecture*. As depicted in Figure 1(a), dedicated computational resources are allocated for each stage separately. The generation stage employs existing inference frameworks like vLLM [23] to generate samples, which are subsequently transferred to the training stage that uses training frameworks like DeepSpeed [8] or Megatron-LM [41]. Updated model weights are then transferred back to the inference framework for the next iteration generation. This architectural choice can effectively reuse existing infrastructures and facilitate rapid deployment of various RL algorithms, which gains broad initial adoption. However, a notable drawback of disaggregated architecture is resource idleness arising from serial dependency: GPU resources allocated to the training stage remain idle during the generation stage, and vice versa.

To address this inefficiency, recent RL training frameworks, such as verl [38], ReaL [28], and RLHFuse [59], have adopted a *colocated architecture*. As illustrated in Figure 1(b), it colocates the generation and training stages on the same GPU resources. When one stage is active, the states of the other stage (such as model weights and optimizer states) is

temporarily stored in CPU memory. Context switching happens between stage boundary, enabling time-division multiplexing of GPU resources. This architectural shift resolves the resource idleness issue and improves training efficiency. Subsequently, colocation becames the prevailing choice and was widely acknowledged to be superior to disaggregation.

We initially believed the same and chose colocated architecture for our internal RL framework. However, in practical deployment, we have observed that the colocated architecture encounters the problem of *resource coupling* as the training scales out. The reason lies in that the two stages feature fundamentally distinct workloads: the generation stage is notably memory-bandwidth-bound [57], whereas the training stage is typically compute-bound [22]. However, due to colocation, both stages must share identical *resource quantities* and *hardware types*, creating an inherent conflict with their divergent computational characteristics.

Concretely, the performance speedup for the generation stage reaches a plateau much more quickly compared to the compute-bound training stage when scaling out resources. Yet, the colocated architecture restricts resource quantities for both stages to be identical, thus diminishing overall resource utilization. Also, it is unable to select the most suitable and cost-effective hardware types for each stage respectively. Moreover, constrained by factors such as policy, cost, and power supply, constructing a single large-scale datacenter can be challenging and expensive [1]. Consequently, companies typically operate multiple medium-sized datacenters equipped with GPUs spanning different generations and types, forming a cross-datacenter heterogeneous resource pool [3]. As RL training scales out, the colocated architecture struggles to efficiently leverage this entire resource pool, as the training stage typically involves full-mesh communication operations [42], which will incur significant communication overhead across datacenters.

Under such circumstances, the initial disaggregated architecture reshines. First, resource quantities for the generation and training stages need not be identical, allowing more flexible and efficient resource allocation. Second, it allows for selecting the most suitable hardware for each stage, such as adopting more cost-effective inference GPUs for the generation stage, while using more expensive and high-performance GPUs exclusively for the training stage. Moreover, RL features point-to-point data transfer between the two stages, keeping the communication overhead manageable even across datacenters. By putting the two stages in separate datacenters, RL training can overcome the constraints of a single homogeneous datacenter and fully leverage the entire cross-datacenter, heterogeneous resource pool.

Nevertheless, the disaggregated architecture encounters two challenges in existing frameworks, preventing it from fully unleashing its potential. The first is the resource idleness as illustrated in Figure 1(a). Naive disaggregation introduces pipeline bubbles due to the serialized execution of the two stages. The second challenge, irrespective of the underlying architecture, stems from the long-tail output length distribution inherent in LLM inference workload [59]. In the later phase of generation, only a small number of long-tail samples remain in the system, leading to severely underutilized GPUs. This problem is further exacerbated by the widespread use of long chain-of-thought generation in modern RL training for reasoning models [13].

To address the above problems, we present StreamRL, an RL training framework specifically designed for the disaggregated architecture. The key insight of StreamRL is abstracting the generation and training stages into *stream generation service* (`SGS`) and `Trainer`, respectively. `Trainer` submits generation requests to `SGS`; once the `SGS` receives prompts and starts generation, it returns each completed sample to `Trainer` in a stream fashion, allowing any follow-up actions of `Trainer` and reducing resource idleness. With streaming, StreamRL can enhance existing naive solutions such as mini-batch pipelining [27] to enable more flexible and efficient concurrent execution of `SGS` and `Trainer`, and can achieve fully overlapping execution under asynchronous RL.

Under this streaming architecture, careful resource allocation between the two stages is necessary to balance their execution time, otherwise pipeline bubbles may still occur. StreamRL utilize a *profiler-based resource allocation algorithm* to decide the resource allocation before training. Furthermore, some recent progress in reasoning LLMs [13] has witnessed improvements in model capabilities during RL training as the LLM output lengths increase. Given that the workload change sensitivity of the two stages differs, StreamRL also provides a *dynamic resource adjustment mechanism* to elastically maintain balanced execution between the two stages throughout the training process.

To address the long-tail issue, StreamRL leverages an *output-length ranker model* to identify long-tail samples, then utilizes a *skewness-aware scheduling mechanism* that selectively allocates resources to long-tail samples and adjusts the batch size to reduce overall generation latency.

Experiments on various LLMs and real-world dataset show that StreamRL achieves up to 2.66× throughput compared to existing state-of-the-art systems, and can further improves up to 1.33× in cost-effectiveness under heterogeneous, cross-datacenter setting.

In summary, we make the following contributions:

- We analyze critical scalability and efficiency issues inherent in current colocated RL frameworks, and propose to revisit the disaggregated architecture.
- We present StreamRL to effectively mitigate pipeline bubbles and the long-tail issue to fully unleash the potential of disaggregated architecture.
- We conduct extensive experiments to evaluate StreamRL's performance against current state-of-the-art RL frameworks and demonstrate its effectiveness in heterogeneous, cross-datacenter scenarios.

2 Background and Motivation

2.1 Background

RL for LLMs. We first explain how key concepts in RL are applied to LLM training. The LLM to be trained works as the *agent model* in the RL semantics, which learns from feedback from interactions with the environment. The prompt represents the *initial state* in which the agent model resides. Each token generated by the agent model is treated as an *action*, which leads to a new state—namely, the combination of the prompt and all the generated tokens. The agent model continues to take actions in each state, autoregressively generating tokens until completing a sample, which is referred to as a *trajectory* or *rollout*. Subsequently, the *reward* is derived from the samples and used to train the agent model.

Concretely, each RL iteration consists of two primary stages: generation and training. In the generation stage, for each given prompt, the agent model first processes all the tokens through one forward pass to generate the first output token, which is called the prefill phase. It also establishes the key-value cache [12, 33], which stores per-token intermediate states. Then, in the decoding phase, the model autoregressively generates subsequent tokens, reusing the key-value cache to avoid redundant computations. Upon completion, each prompt, combined with the generated tokens, forms a single training sample. For sample efficiency, hundreds of samples are generated in batch for each iteration.

In the training stage, generated samples are scored with reward. Depending on the algorithm, reward computation can be performed by an additionally trained *Reward Model* [9] or by rule-based functions [13]. The latter approach is commonly used for tasks with explicit correctness criteria, such as coding and solving maths problems, and recent research has demonstrated its effectiveness in improving model's reasoning ability [7, 13, 53]. In addition to coarse-grained, sample-level reward, certain RL algorithms, such as PPO [36], introduce a *Critic Model* to provide fine-grained, action-level reward for each token. In the constrast, other algorithms like GRPO [37] use approximation strategies to avoid using Critic Model. To enhance training stability, a *Reference Model*—initialized from the before-trained agent model and remained frozen during training—is used to provide Kullback-Leibler (KL) divergence regularization. It prevents the agent model from deviating much during training. The final loss incorporates both sample- and token-level rewards along with KL divergence to train the agent model. After updating parameters, it proceeds to the next iteration's generation.

LLM Parallelization. To scale up LLM training, several parallelization techniques have been developed. Data Parallelism (DP) involves duplicating the model across devices and splitting the dataset among them, allowing each replica to process different portions of data concurrently. After each training step, gradients must be synchronized among all replicas. Tensor Parallelism (TP) divides individual operations across multiple GPUs, with each GPU responsible for a portion of the computation. Due to its high communication overhead, TP is typically confined to intra-node deployment where high-speed interconnects like NVLINK are available. Pipeline Parallelism (PP) splits the model layers into separate stages, assigning each to a different device or node. The input batch is further divided into microbatches, enabling pipelined execution and full-batch gradient accumulation. Since each method offers distinct trade-offs, modern training systems [22, 40] often combine all three approaches to maximize scalability and efficiency.

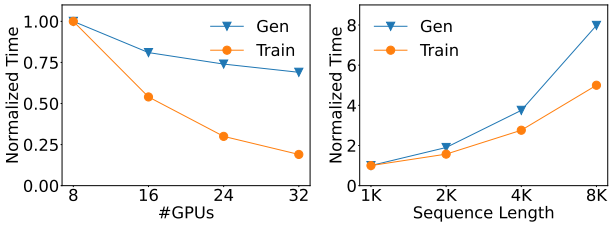

Figure 2. The performance sensitivity difference of the generation and training stage under resource quantities (left) and sequence length (right).

2.2 Problems with Colocation

As mentioned in §1, the colocated architecture for RL framework, with its advantage in resource efficiency, appears to be a better choice. We also held this belief at the outset and built our internal RL training framework based on colocated architecture to support the large-scale training of commercial models. However, with real-world deployment over time, as model size and training scale continues to grow, we found that the intuitively better colocated architecture began to reveal its fundamental limitation, i.e., *resource coupling*.

In the colocated architecture, the generation and training stages must share the same set of devices. However, the fundamental issue lies in the fact that these two stages represent fundamentally different workloads. During the decoding phase of the generation stage, each sample computes only on the newly generated token from the previous step, but still requires access to the full set of model parameters, making it highly memory-bandwidth-bound. In contrast, the training stage is compute-bound, as both forward and backward passes compute over all tokens in the batch simultaneously, easily saturating the GPU's compute units. Due to colocation, both stages must share identical *resource quantities* and *hardware types*, creating an inherent conflict with their divergent computational characteristics.

Resource quantities. Given the workload, we expect the iteration time to decrease with more resources. However, due to the distinct computational characteristics of the generation and training stages, their scaling sensitivity to resource quantities differ significantly. We profile the execution latency of each stage under different resources when training

	H20	H800
BF16 TFLOPS	148	**989.5**
HBM capacity	**96GB**	80GB
HBM bandwidth	**4TB/s**	3.35TB/s
NVLINK bandwidth	**900GB/s**	400GB/s
Cost per machine [60]	1.00	**2.85**

Table 1. NVIDIA GPU specifications.

a 7B LLM on a fixed workload with a sequence length of 8192. As shown in the left of Figure 2, generation time quickly reaches a plateau as resources increase. This is because, being memory-bandwidth-bound, the generation time is mainly determined by the overall memory bandwidth. Among parallelism strategies, only increasing the tensor parallelism (TP) size can effectively increase the overall bandwidth. However, due to the high communication overhead of TP, it is typically limited to intra-node where NVLINK are available. As a result, scaling the resources for generation stage mainly increases the number of generation instances, i.e., the DP size, which has limited effect on reducing generation time.

In contrast, since the training stage is compute-bound, it benefits much more from resource scaling, achieving better acceleration. The consequence of this difference in scaling sensitivity is that when scaling up resources, the generation stage suffers from low resource utilization, as increased resources do not translate into proportional performance gains. This issue becomes increasingly prominent as long chain-of-thought generation grows in importance, leading to longer model outputs and a rising share of generation time in the overall iteration time.

Hardware types. Another manifestation of resource coupling lies in hardware selection. As shown in Table 1, different NVIDIA GPU types exhibit trade-offs between compute capability, memory bandwidth and cost. Some GPUs, such as H20, are specifically designed for memory-bandwidth-bound workloads like inference, offering even higher HBM bandwidth and larger HBM capacity than flagship ones like H800, while costing only about 35% as much. Therefore, from the perspective of training cost —e.g., throughput per cost—the colocated architecture prevents selecting the most cost-effectiveness hardware for each stage.

2.3 Motivations for Disaggregation

In contrast, the initial disaggregated architecture exhibits several unique advantages that deserves reconsideration.

Flexibility. Under the disaggregated architecture, the resource coupling problem mentioned above is immediately eliminated, allowing dedicated resource allocation tailored to the distinct workloads of the two stages. Additionally, it enables selecting the most suitable hardware for each stage, flexibly leveraging heterogeneous resources to improve training cost and overall cost-effectiveness.

Scalability. Due to various constraints, many organizations operate multiple medium-sized datacenters instead of one monolithic and giant datacenter. As training scales out, As training scales out, cross-datacenter training becomes increasingly appealing if it is feasible. Traditional LLM training, however, involves extensive full-mesh communication operations, imposing high bandwidth demands on networking and making cross-datacenter deployment challenging. In contrast, the disaggregated RL architecture requires relatively low inter-stage communication. Generated samples and related metadata are small in size, and although model weights must be transmitted, they only require point-to-point transmission instead of full-mesh network topology. This is well-suited for inter-datacenter dedicated links, making cross-datacenter RL training practically feasible rather than merely theoretical. Furthermore, generation instances are entirely independent, allowing them to be distributed across multiple datacenters, thus fully utilizing the entire resource pool and scaling out.

2.4 Challenges for Disaggregation

Although the disaggregated architecture seems natural and promising for the two-stage RL workflow, fully unlocking its potential and surpassing the performance of existing colocated architectures requires addressing several challenges. As shown in Figure 3, the training timeline under naive disaggregation reveals two types of bubbles that lead to GPU under-utilization.

Pipeline bubbles. This is the primary source of inefficiency in existing disaggregated frameworks. The generation stage sends samples to the training stage only after all samples have been generated, during which time the resources allocated to the training stage remain idle. Similarly, when the training stage is active, the generation stage's resources are also left unused, waiting for the up-to-date model weights.

Furthermore, traditional LLM training is a relatively static workload, whereas in RL training, samples are generated online and thus exhibit dynamic behavior. As noted in the DeepSeek-R1 technical report [13], the LLM will spontaneously increase its generation length over time, enhancing its reasoning ability through self-reflection—an effect referred to as *inference-time scaling* [2, 13]. However, the generation and training time respond differently to such workload changes. As shown on the right side of the Figure 2, with increasing sequence length, generation time grows more significantly than that of training. This is primarily due to the enlarged key-value cache size, which, to avoid out-of-memory (OOM), forces smaller batch sizes during generation and consequently reduces GPU utilization. To better address pipeline bubbles under a disaggregated architecture, it is desirable for the execution time of the two stages to be closely matched so that some overlapping techniques like mini-batch pipelining [27] and asynchronous pipelining [27]

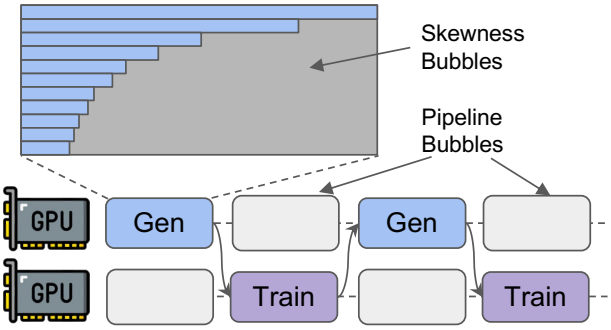

Figure 3. Resource waste in disaggregated architecture.

can be applied (§4.1). However, the differing growth in latency under dynamic workloads leads to stage imbalance, introducing new bubbles.

Skewness bubbles. Another type of bubbles originates from the RL workload itself. In the generation stage, the output length has a skewed distribution [58], with only a small subset being much longer than the majority. As generation proceeds, only a small set of long-tail samples remain in the system. This undermines GPU utilization because the decoding phase of generation requires a large batch size—often in the hundreds—to maintain high throughput for its memory-bandwidth-bound nature. Making the problem even worse, under the guidance of inference-time scaling [2, 13], the output length continues to grow, making the skewness bubbles an urgent problem in real-world deployment.

An engineering workaround [43] is to temporarily store partially generated long-tail samples in a replay buffer and generate part of them in each iteration. However, this changes the original output length distribution and may negatively impact model quality, introducing a trade-off between accuracy and efficiency. Another solution, built upon the colocated architecture, is adopted by RLHFuse [58]. It compacts the long-tail samples onto a small subset of resources and uses the freed-up machines to pre-execute part of the training stage's work—such as KL divergence computation and reward derivation—alongside the generation of long-tail samples, effectively filling the skewness bubbles. However, this approach is not applicable in the disaggregated architecture, where the two stages are physically separated.

3 StreamRL Overview

To this end, we present StreamRL, an efficient RL framework designed with disaggregation from first principle. As shown in Figure 4, StreamRL abstracts the generation and training stages into Stream Generation Service (`SGS`) and `Trainer`, respectively. `SGS` and `Trainer` are deployed on physically separate resources, potentially even in different datacenters connected by a point-to-point link. This architectural design fully unleashes the benefits of disaggregation discussed in §2.3, enabling (1) *flexible resource allocation*, (2) *heterogeneous hardware selection*, and (3) *cross-datacenter training*.

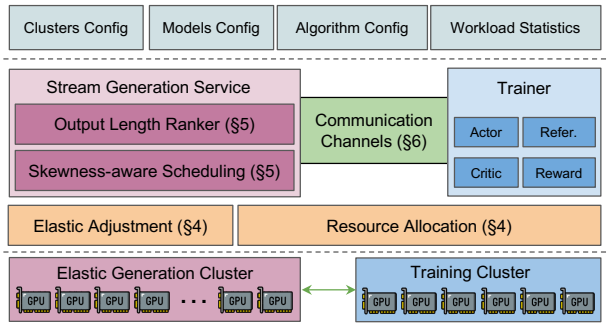

Figure 4. StreamRL system architecture.

We first present a high-level overview of the overall workflow of StreamRL. Next, we describe in detail our techniques and designs for addressing pipeline bubbles (§4) and skewness bubbles (§5), as well as the implementation details of the communication between `SGS` and `Trainer` (§6).

Workflow. Given the clusters, models, and algorithm configurations, StreamRL first determines how to allocate resources between `SGS` and `Trainer`, as well as which parallelization strategies to adopt for each.

During training, `SGS` exposes two external APIs to `Trainer`: `update(weights)` and `generate(prompts)`. `Trainer` address pipeline bubbles by adjusting the timing of weights updates and handling the early streamed-back samples according to the specific RL algorithms. To address skewness bubbles, `SGS` utilizes an output length ranker to identify long-tail samples. Based on the predictions, it dispatches prompts to specific generation instances and decides scheduling order accordingly, effectively mitigating the bottleneck caused by long-tail samples.

Although the static configuration ensures the balance between generation and training time at the start of training, the dynamic nature of RL workloads requires elastic resource adjustment to maintain close execution time between the two stages throughout the training process. To achieve this, `SGS` continuously monitors `Trainer`'s execution time. As the workload evolves and sequence length increases, if the generation time exceeds the training time by a certain threshold, `SGS` automatically scales out by increasing its DP size to maintain dynamic balance in execution time.

4 Tackle Pipeline Bubbles

4.1 Overlapping Design

To address pipeline bubbles, the key is to ensure that the training stage remains active while generation is ongoing. Mainstream RL algorithms can generally be categorized into two types: synchronous and asynchronous, depending on whether the training samples are generated using the latest model weights. For each type, we present the current strawman solutions and describe how the efficiency of overlapping can be further improved with the support of streaming.

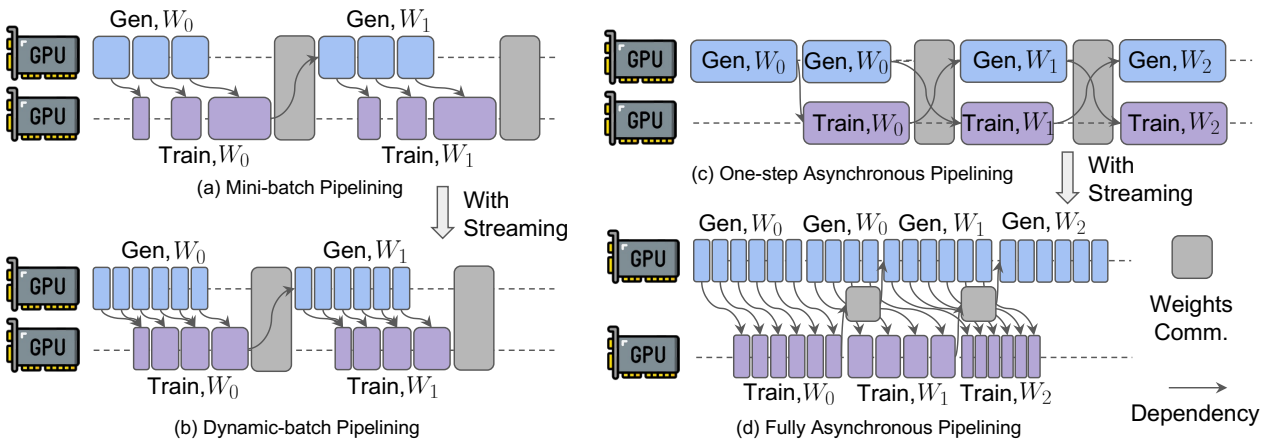

Figure 5. How streaming powers existing solutions to better mitigate pipeline bubbles. W_i denotes the parameter version.

Strawman solution 1: Mini-batch pipelining [27]. In synchronous RL, weights update happens after all samples have been processed. As shown in Figure 5(a), the samples can be evenly divided into several mini-batches analagous to pipeline parallel. Once the number of generated samples reaches the size of a mini-batch, they are passed to the training stage for processing. This approach requires manually setting the mini-batch size: if set too large, it reduces the effectiveness of overlapping; if too small, it harms training efficiency. In practice, the mini-batch size is set empirically to a suitable constant [27]. Nevertheless, due to the long-tail effect, the sequence lengths of the later mini-batches gradually increase. As a result, the training of the last few mini-batches often spill over after generation, creating significant pipeline bubbles [27]. Also, due to the imbalance between mini-batches, it is hard to set the mini-batch size to avoid idle time in the training stage.

Our solution: Dynamic-batch pipelining. We propose replacing the current batched generation with *stream generation*, where samples are immediately sent to the training stage as soon as completed. This enables sample-level operations such as Reference Model inference, KL loss computation, and reward calculation to begin without delay. As shown in Figure 5(b), the training stage can start as soon as it receives enough samples to saturate the GPUs, enabling dynamic batching based on the generation speed. This eliminates idle time in the training stage except for the first mini-batch and effectively reduces the bubbles caused by the last few mini-batches.

Strawman solution 2: One-step asynchronous pipelining [27]. Essentially, the two stages in synchronous RL still work on the same batch of samples, so the serialized dependency within each iteration remains, making it impossible to achieve perfect overlapping. Recently, many works [27, 31, 43, 48] have explored *off-policy asynchronous RL*, where the samples used for training are not necessarily generated with the up-to-date weights, allowing for some extent of staleness. Existing studies [31, 48] and our experiments 7.4 show that one-step asynchronous RL for LLMs does not compromise model performance or convergence.

As shown in Figure 5(c), we can first generate one additional batch while the training stage processes samples from the previous iteration, thereby shifting the dependency to a cross-iteration manner and achieving better overlapping. However, the issue with this batch-level pipelining is that each iteration still ends with a global synchronization to transmit the weights, during which both stages remain idle. Moreover, due to the dynamic nature of online generation, there can be fluctuations in generation and training time across iterations, which are difficult to accurately align with resource adjustments, resulting in new bubbles.

Our solution: Fully asynchronous pipelining. As shown in Figure 5(d), the above issues can be resolved with streaming. First, weight transmission can overlap with the training of the next iteration, since samples from the previous iteration have already been streamed and buffered for training. Meanwhile, the generation of the current iteration does not depend on the latest weights, and can also proceed in parallel. This removes weight transmission completely from the critical path. Moreover, even if there are fluctuations in generation and training time across iterations, as long as their average speeds are matched and the fluctuation is limited, no new bubbles will emerge. Note that here we do not introduce asynchronous samples beyond one step, therefore the training semantics remain identical to the naive solution.

4.2 Stage Balancing

To achieve better overlapping between `SGS` and `Trainer`, we need to carefully balance the execution times of the two stages. As a result, deciding the appropriate parallel strategies and number of GPUs for each stage becomes critical for minimizing overall iteration time.

Algorithm 1 Resource Allocation Algorithm.

Input: GPU budget, profiler-based estimation model $\mathcal{P}$, training workload $\mathcal{W}$.
Output: GPU allocation for SGS and Trainer $(x_{\text{opt}}, y_{\text{opt}})$.

```
Single-datacenter: total GPU budget n
T* = ∞
for each configuration (x, y) where x + y ≤ n do
    T_gen = P_gen(x, W), T_train = P_train(y, W)
    T_latency ← max(T_gen, T_train)
    if T_latency < T* then
        T* ← T_latency, x_opt, y_opt ← x, y
    end if
end for
return x_opt, y_opt
Cross-datacenter: respective GPU budget m, n
T_gen = P(m, W), T_train = P(n, W)
if T_gen < T_train then
    Find k s.t. |P_gen(k, W) − T_train| achieves minimum.
    return k, n
else
    Find k s.t. |P_train(k, W) − T_gen| achieves minimum.
    return m, k
end if
```

Parallel configuration. Before deciding how many resources to allocate to each stage, we first address a subproblem: determining the optimal execution time of either SGS or Trainer under a given workload and GPU budget. This essentially reduces to optimizing the parallel strategy, which is a well-studied problem for both LLM training [54, 55] and generation [57]. For Trainer, we adopt a profiler-based approach inspired by prior work on automated parallelism [29, 47, 54, 55]. Due to the determinism of DNN execution time [17], we can accurately model training time under a fixed GPU budget with minimal profiling. For SGS, generation time depends on the scheduling strategy during inference [26]. Fortunately, under our skewness-aware scheduling (§5.3), generation time is also deterministic for a given workload, allowing us to model it similarly. Note that we assume access to the RL workloads. In practice, this can be obtained from samples generated by recent training iterations or bootstrapped from samples generated by the LLM prior to training.

Resource allocation. Building on the above strategies, we can determine the resource allocation for each stage. StreamRL supports two deployment solutions in production RL training. The first is single-datacenter deployment, where SGS and Trainer are colocated within the same datacenter equipped with homogeneous hardware resources. This is the standard setup in prior LLM training systems [22, 41]. Also, StreamRL supports cross-datacenter deployment, leveraging the decoupled nature of SGS and Trainer in RL workflows and placing SGS and Trainer in separate datacenters with heterogeneous hardware (e.g., H20 vs. H800). We present the resource allocation algorithms under two different deployments as shown in Algorithm 1.

Single-datacenter. We define the number of GPUs allocated to SGS and Trainer as x and y, respectively. The resource constraint is $x + y \leq n$, where n denotes the total GPU budget. To determine the optimal allocation strategy, we enumerates the allocation configurations. For each case, we can get the generation and training time respectively using the aforementioned profiler-based modeling. Then we use the larger of the two as the estimated latency and selects the optimal (x, y) pair that minimizes overall iteration time.

Cross-datacenter. For cross-datacenter deployment, let m and n denote the available GPUs in the datacenters for SGS and Trainer deployment, respectively. The resource constraints are $x \leq m$ and $y \leq n$, making x and y independent variables. A naive choice is $(x, y) = (m, n)$, which fully utilizes all the resources in both datacenters. However, since iteration time is determined by the slower stage of SGS and Trainer, such full allocation may lead to resource waste. To address this, we identify the faster stage under full allocation and gradually reduce its GPU usage until both stages achieve similar execution times. This strategy eliminates unnecessary GPU usage, allowing surplus resources to be reallocated to other jobs within the respective datacenter.

Dynamic adjustment. The above techniques only ensure that the two stages are balanced at the beginning of training. As observed in the DeepSeek-R1 technical report [13], the generation length of LLMs increases progressively during RL training, leading to evolving compute and memory demands for both SGS and Trainer stages. Unfortunately, the two stages exhibit different sensitivities to workload changes in terms of latency. To address this, we propose a dynamic adjustment mechanism that monitors the execution time gap between generation and training, denoted as δ. As shown in Figure 2, generation time increases faster than training, which means δ will gradually increase as training proceeds.

Ideally, when δ exceeds a certain threshold, we can rerun the resource allocation algorithm to rebalance the stages. However, in practice, all the GPUs in Trainer are tightly coupled through communication group due to the 3D parallelism. Changing the parallel strategy or reallocating resources for Trainer requires restarting the entire training runtime, which incurs significant overhead. In contrast, generation instances in SGS are naturally decoupled. Therefore, StreamRL estimates the reduction in generation time, δ', achievable by adding one data parallel (DP) unit to SGS. δ' is calculated using the aforementioned profiler and the current RL workload. When $\delta \geq \delta'$, adjustment is triggered by adding one more DP unit to SGS. This reallocation does not interrupt training, and the overhead is limited to initializing

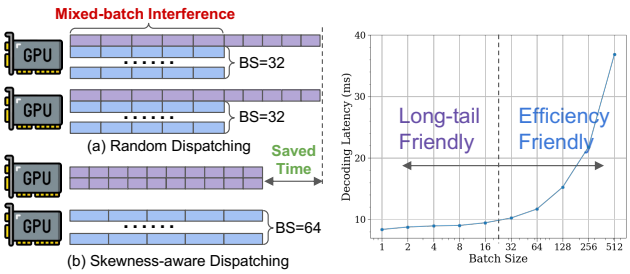

Figure 6. Left: The advantage of skewness-aware dispatching over random dispatching. Right: The trend of per-token decoding latency for a 7B LLM profiled on NVIDIA H800 with vLLM [23] as the batch size increases.

the added DP unit, which is negligible relative to the overall RL training time.

5 Tackle Skewness Bubbles

In this section, we introduce the techniques used by SGS to tackle skewness bubbles. At a high-level, SGS minimizes the generation time under given resources. which serves as a function used by the resource allocation section (§4.2).

5.1 Problems and Opportunities

Problem 1. Existing systems do not differentiate between long-tail samples and regular samples. To achieve workload balance, prompts are typically assigned randomly across generation instances. Figure 6(a) shows an simple example where the generation DP size is 2, with 2 long-tail samples whose output length are two times that of the remaining 64 samples. Under the random dispatching strategy, each generation instance receives one long-tail sample along with half of the regular samples. This results in significant interference for the long-tail samples during the first half of generation due to batched inference.

The right side of Figure 6 shows the trend of per-token decoding latency for a 13B model on an NVIDIA A100 GPU as the batch size increases. It can be observed that latency grows slowly before reaching compute-bound, and then increases almost linearly after that. To improve throughput, existing systems usually accumulate a sufficiently large batch size for each instance through random dispatching. However, in the presence of long-tail samples, this approach not only slows down their decoding in the early stages but also leads to extremely low utilization in the later stage, as only a few long-tail samples remain in the system.

Opportunity 1. With an intuitive understanding of the problem, we now proceed to the solution. The generation latency of a sample can be modeled as:

$$Sample\ Latency = PTL(BS) \times L \tag{1}$$

where per-token latency (PTL) is a function of batch size (BS) which can be profiled in advance and L is the output length. It can be observed that the random dispatching strategy balances load merely based on L, without considering PTL. For samples with longer L, we actually prefer to reduce their PTL—that is, their BS—since PTL is a monotonically increasing function of BS.

The heuristics for the solution naturally emerge: we can extract the long-tail samples and assign them to a few dedicated instances with smaller batch sizes, allowing them to decode at the fastest possible speed and eliminating interference caused by batching. Meanwhile, regular samples can be grouped into large batches to fully utilize GPU resources. By extending the original one-dimensional load balancing into a two-dimensional scheme, generation latency can be effectively reduced, as illustrated in Figure 6(b).

Problem 2. The above approach relies on a key assumption: that long-tail samples can be identified before generation begins. However, the output length of LLM generation is typically regarded as *not known a priori*.

Opportunity 2. Fortunately, while it is difficult to predict the exact generation length of each sample, it is possible to estimate the *relative ranks* of output lengths [14] with another model. Intuitively, the ranking problem is essentially a classification problem, as more difficult prompts typically require more reasoning—i.e., longer output lengths. Therefore, the ranking model essentially classifies prompts based on their difficulty. Difficulty is a property inherent to the prompts themselves, which can be generalized across different LLMs, leading to relatively high prediction accuracy. In contrast, the exact output length is a characteristic of the LLM itself and is significantly more challenging to predict. Our experiments (§7.2) show that the top 20% long-tail samples can be recalled with nearly 90% accuracy, which aligns well with our hypothesis. Since the generation time is predominantly bottlenecked by the long-tail samples, accurately identifying them is sufficient to achieve the majority of the performance gains compared to the upper bound speedup where the oracle is available (§7.2).

Next, we detail how the above observations are incorporated into the design of the output length ranker (§5.2) and the skewness-aware scheduling (§5.3).

5.2 Output Length Ranker

Method. To train the ranker model, we collect a set of input prompts along with their corresponding output lengths from the target LLM. These (prompt, length) pairs can be sourced from our online inference serving service or generated offline before training. We then concatenate the prompts with their corresponding output lengths to form a training dataset. Using this dataset, we directly perform supervised fine-tuning (SFT) on a small LLM as the ranker model.

After fine-tuning, the ranker model can take a batch of prompts as input and estimate their absolute output lengths.

Algorithm 2 Skewness-aware Dispatching Algorithm.

Input: Batch of prompts $\mathcal{P}$, estimated lengths $\mathcal{L}$, longtail threshold α, output length distribution $\mathcal{D}$, and N generation instances

Output: Number of DP instances for long-tail and regular samples respectively.

```
P ← Sort(P, L, descending)
P_α ← P[: α × |P|]                    ▷ Long-tail samples
P_r ← P[α × |P| :]                    ▷ Regular samples
L_α ← P90(D), L_r ← P50(D), L* ← ∞
for N_l, N_r such that N_l + N_r = N do
    L_total_latency ← L_latency(P_α, L_α, N_l) + L_latency(P_r, L_r, N_r)
    if L_total_latency < L* then
        L* ← L_total_latency, N_l* ← N_l, N_r* ← N_r
    end if
end for
return (N_l*, N_r*)
```

These estimated lengths are then used to sort the prompts, producing the final ranking result. Note that the SFT process involves predicting absolute lengths. As RL training progresses and the parameters of the target LLM evolve, even the same prompt may yield different output lengths. Therefore, after a period of training, we perform online fine-tuning of the ranker model using recent generation results, following the same methodology.

Fortunately, the difficulty of a prompt is an inherent property and remains stable. As a result, even if the absolute predictions drift, the relative ranking produced by the ranker remains reasonably accurate, reducing the need for frequent online fine-tuning.

Overhead. One concern is the overhead introduced by the ranker model. First, the ranker model trains very quickly, requiring only a few minutes to converge. After training, we perform a one-time offline preprocessing step on the dataset used for RL, estimating the actual output length for each prompt. These estimates serve as the basis for the subsequent skewness-aware scheduling. Note that this preprocessing is conducted entirely offline, meaning the ranker model imposes no online overhead on the RL training process and does not affect the original training efficiency.

5.3 Skewness-aware Scheduling

With the help from output length ranker, SGS will receive a batch of prompts along with their estimated output lengths. It then needs to make two decisions: determine how to dispatch the prompts to different generation instances, and decide the scheduling order within each instance after dispatching.

Dispatching. Given a batch of prompts, we first sort them by their estimated output lengths from longest to shortest. With the relative order established, we mark the longest $\alpha\%$ of them as long-tail samples, where α is a hyperparameter; in practice, setting α to 20 yields good results (§7.4). Next, we need to select N_l out of N generation instances to handle the long-tail samples, while the remaining N_r instances are used for regular samples.

To ensure workload balance, we need an estimate of the actual workload for the regular and long-tail samples. Here, we assume access to the output length distribution $\mathcal{D}$ of the LLM to be trained. This workload characteristic can be derived from recently generated samples during training or by having the LLM generate a set of samples beforehand for bootstrapping. We use the P50 and P90 of $\mathcal{D}$ to estimate the average output lengths for regular and long-tail samples, respectively. Based on this, we extend the sample latency (1) to estimate the single instance generation latency:

$$Latency = PTL(BS) \times L_{avg} \times \lceil \frac{M}{BS} \rceil \tag{2}$$

where L_{avg} is the estimated average output length of the samples assigned to the instance, depending on whether the instance processes regular or long-tail samples, and M is the number of prompts assigned to the instance. Ideally, we would like $BS = M$, so that all samples can be processed in a single round. However, with longer output lengths, a larger BS will result in higher key-value cache memory usage, so BS is constrained by the GPU memory capacity.

With Equation 2, we can iterate through all (N_l, N_r) configurations and find the one that minimizes the generation time. Algorithm 2 shows the pseudocode for the skewness-aware dispatching algorithm. After that, the long-tail and regular samples can be evenly distributed within their respective instances.

Scheduling Order. As mentioned earlier, BS is limited by key-value cache memory usage, so the number of samples M assigned to each instance may exceed the BS used during generation. In this case, multiple rounds of generation are required, which introduces the need for deciding the scheduling order of samples. This problem is a variant of the $P||C_{\max}$ (makespan minimization) problem, where P denotes parallel processing units and $C_{\max}$ is the maximum completion time. In our case, each generation iteration of batch size BS is viewed as BS parallel processing units. We employ a well-known greedy algorithm, longest-processing-time-first (LPT) scheduling [15], to address this problem. Specifically, samples are assigned to the batch in descending order of their estimated output lengths. Once a sample is completed, the sample with the longest remaining output length is added to the batch. This process continues until all samples are processed. Prior work [15] has proved that LPT scheduling is 4/3-approximation, i.e., its completion time is at most 4/3× that of the optimal scheduling.

Models	# of Layers	# of Q Heads	# of K/V Heads	Hidden Size
Qwen2.5-7B	28	28	8	3584
Qwen2.5-32B	64	40	8	5120
Qwen2.5-72B	80	64	8	8192

Table 2. LLM specifications.

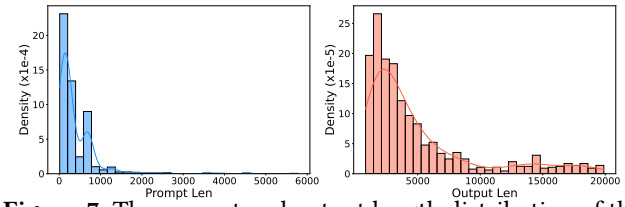

Figure 7. The prompt and output length distribution of the evaluation dataset.

6 Implementation

RL Training Framework. SGS employs an in-house inference engine implemented in C++ with optimized CUDA kernels, supporting continuous batching [52] to release shorter samples early and prefix sharing [56] to save key-value cache usage. `Trainer` implements 3D parallelism similar to prior work [22, 28, 41]. To address GPU memory constraints, we develop dynamic CPU offloading that interleaves the execution of different models through memory swapping.

Tensor-native RPC Library. Conventional distributed computing frameworks [30, 44] typically incur significant serialization and deserialization costs for tensor data transfers. We developed RL-RPC, a communication framework optimized for efficient data transfer between SGS and `Trainer`. The system employs GPU-Direct RDMA for zero-copy tensor transfers, bypassing CPU involvement and eliminating serialization overhead to minimize communication costs. By fully leveraging RDMA bandwidth without consuming GPU SM resources, RL-RPC prevents performance degradation through overlapping of communication and computation. A TCP fallback mechanism ensures compatibility across different network environments, including non-RDMA cross-datacenter connections.

Weights transmission. After trainer-side weights sharding, StreamRL employs a network-aware transmission engine to efficiently broadcast weights from `Trainer` to SGS. The engine dynamically builds broadcast trees optimized for network topology. In the single-datacenter setting, where both reside on the same RDMA network, it creates multiple trees rooted at different DP ranks, load-balancing across trees to keep all DPs bandwidth-saturated. For cross-datacenter deployment with limited connection bandwidth, only the root (DP rank 0) sends weights to a desinated SGS DP instance in the remote datacenter, followed by a local broadcast to minimizes cross-datacenter traffic.

7 Evaluation

In this section, we evaluate StreamRL with LLMs of different sizes ranging from 7B to 72B on real-world dataset. First, we compare the end-to-end performance of StreamRL to other RL training frameworks under single-datacenter setting (§7.1), and conduct ablation studies to show the effectiveness of our proposed techniques (§7.2). Next, we show the performance of StreamRL under heterogeneous, cross-datacenter setting to demonstrate the flexibility and scalability of disaggregated architecture (§7.3). Finally, we provide training curves to demonstrate that asynchronous RL achieves comparable performance and convergence to synchronous RL (§7.4).

Testbed. We deploy StreamRL on a H800 cluster with 16 nodes and 128 GPUs. Each node has 8 NVIDIA H800-80GB GPUs. Nodes are connected by 8 * 200 Gbps RDMA network based on RoCEv2 with rail-optimized topology. For the heterogeneous and cross-datacenter experiments (§7.3), we also utilize a cloud-based H20 cluster with 4 nodes and 32 GPUs. Each node has 8 NVIDIA H20-96GB GPUs. Nodes are connect by 100Gbps TCP network. The H800 and H20 clusters are connected by a 80Gbps dedicated link. Other specifications between H800 and H20 are listed in Table 1.

Models. We choose Qwen2.5 models [5] ranging from 7B to 72B, which is a popular base model family used for RL post-training both in academia and industry. The detailed model architectures are listed in Table 2.

Dataset. We use an internal CodeMath prompts dataset and collect responses from DeepSeek-R1 [13], an open-source, advanced reasoning model, as ground truth. The distributions of prompt length and output length in the dataset are shown in Figure 7. The maximum output length is 20K and the distribution is very long-tail. To train StreamRL's output length ranker model, we split the dataset into training, validation, and test sets with a ratio of 7:2:1. All performance evaluations are conducted on samples from the test set. To avoid discrepancies in generation length caused by numerical differences in underlying runtime across different RL frameworks, we modify the inference code in all RL frameworks to generate outputs with the same length following the ground truth of each prompt. This not only help simulating the long-tail effects observed in real-world RL training while ensuring a fair comparison across different RL frameworks.

Settings. We use the PPO algorithm [36] which is widely used in RL post-training. But note that the effectiveness of StreamRL does not rely on any specific RL algorithm and can generalize to others such as GRPO [37]. We set the Actor, Critic, and Reference Model to the same size. We do not use an explicit Reward Model but a rule-based verifier to provide rewards following [13]. We also proportionally scale down the output lengths by two and four, to simulate the early and middle stages of RL training, when the output lengths are

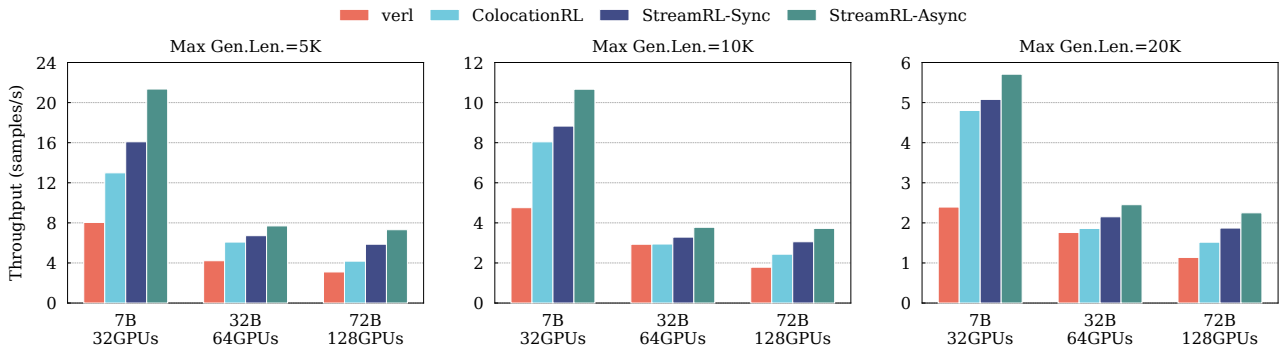

Figure 8. End-to-end throughput of RL training systems under different sequence length and model size settings.

not yet particularly long. This leads to three datasets, which we denote as 5K, 10K, and 20K for clarity. In each iteration, we use a global batch size of 1024 following [38].

Metrics. For the end-to-end experiment, we measure the sample throughput following [58], which is defined as the average number of samples processed per second. Under each setting, we record the sample throughput over 20 consecutive training iterations after warm-up.

Idx	Method	Normalized Throughput
1	Colocation Baseline	1.00
2	(1) with skewness-aware scheduling	1.08 (+8%)
3	(2) with disaggregation + streaming	1.23 (+15%)
4	(3) with asynchronous	1.48 (+25%)

Table 3. Throughput improvement breakdown when training 72B model on the 20K dataset.

7.1 End-to-end Experiments

We compare the end-to-end performance of StreamRL against the following baseline frameworks.

- verl [38] is the state-of-the-art open-source RL training framework and a representative of the colocated architecture. It proposes a hierarchical hybrid programming model for the RL dataflow and optimizes the parallel strategies of each model. We choose vLLM [24] as its inference engine and Megatron-LM [41] as its training backend.
- ColocationRL is our in-house RL training framework based on a colocated architecture. It shares the same inference and training backend implementations as StreamRL. We include this baseline to demonstrate the performance improvements brought by disaggregation and our techniques in §4 and §5, eliminating any unfair comparisons caused by differences in underlying implementations and other optimization techniques in LLM generation and training that are orthogonal to our core contributions.

We do not compare against other open-source frameworks like OpenRLHF [19] and NeMo [18] which are based on disaggregated architectures, as they have lower throughput than verl due to resource idleness (§2.2) as reported in [38]. For StreamRL, we show two variants. StreamRL-Sync implements the synchronous version of PPO, which is the same as the baselines. StreamRL-Async implements the one-step asynchronous version to maximize throughput.

Figure 8 presents the end-to-end throughput of various RL frameworks under different maximum sequence lengths and model sizes. Compared to verl, StreamRL-Sync achieves a 1.12×–2.12× speedup, partially attributed to optimizations in the underlying inference and training framework. Compared to ColocationRL, StreamRL-Sync achieves a 1.06×–1.41× speedup by leveraging disaggregated stream generation and skewness-aware scheduling. Under the Colocation setup, generation is highly memory-bandwidth-bound, leading to low GPU utilization. In contrast, disaggregation enables StreamRL-Sync to flexibly and judiciously allocate resources for generation and effectively overlaps the two stages via streaming, thereby improving GPU utilization. However, even with streaming, the performance gains from disaggregation are partially offset due to the long-tail distribution of data and stage dependencies. StreamRL-Async further addresses these limitations by employing one-step asynchronous training to fully overlap pipeline bubbles, achieving 1.30×–2.66× throughput improvement.

7.2 Ablation Studies

Improvement breakdown. Table 3 shows the detailed improvement breakdown of our proposed techniques on the 72B model under the dataset with 20K maximum length.

We observe that (2) skewness-aware scheduling improves throughput by 8% over ColocationRL, primarily by optimizing generation time. Using the output length ranker model to identify long-tail samples, we accelerate their generation by assigning them dedicated compute resources and smaller batch sizes. The effectiveness of this skewness-aware scheduling depends on the prediction accuracy of the ranker model for long-tail samples. Table 4 shows the recall rates for different proportions of long-tail samples using models trained

Base Model	Tail 20%	Tail 10%	Tail 5%
Qwen2.5-7B	0.87	0.82	0.76
Qwen2.5-3B	0.85	0.79	0.72
Qwen2.5-1.5B	0.81	0.75	0.68

Table 4. Recall rate under different tail rates and base models.

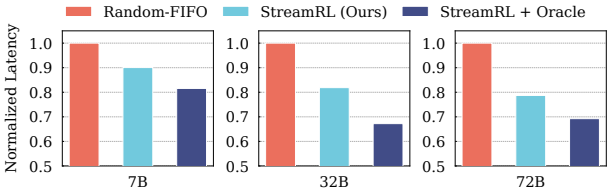

Figure 9. Generation time with different scheduling algorithms on various models under the 20K dataset.

with base models of varying sizes. For the longest 20% of samples, we can achieve the recall rate up to 87%.

How much performance is impacted by the remaining unpredicted long-tail samples? We compare generation time under random dispatching and skewness-aware scheduling, and also evaluate an oracle setting where output lengths are known in advance as the speedup upper bound. As shown in Figure 9, we achieve most of the potential gains with our ranker model. As RL training progresses, the output length distribution of the LLM evolves. To maintain prediction accuracy, we periodically perform online finetuning of the ranker model using recently generated samples to adapt to the distribution shift. The convergence time for this training is just a couple of minutes, which is negligible compared to the overall RL training time.

Building on skewness-aware scheduling, (3) disaggregated streaming further improves throughput by 15%, and (4) asynchronous training yields an additional 25% gain. These improvements fundamentally stem from the ability of disaggregation to allocate appropriate resources to the generation stage, thereby increasing its GPU utilization. To convert this reclaimed utilization into end-to-end speedup, it is necessary to address pipeline bubbles caused by stage dependencies. Streaming overlaps part of the bubbles, while asynchronous training achieves nearly the full overlapping.

Resource allocation. To achieve better overlapping, balancing the latency of the two stages is also critical. Figure 10 compares the iteration time breakdown under a naive evenly-split scheme and the ones selected by StreamRL's resource allocation algorithm. The number of GPUs used for each stage is annotated at the top of each bar. As shown, by adjusting the parallel strategies and resource allocation, we achieve well-balanced stage latencies. In asynchronous training, the iteration time is determined by the slower of the two stages, so balanced stage latencies directly translate into the speedup of 1.25×.

To show the effectiveness of the dynamic adjustment algorithm, we deploy StreamRL-Async on 32 GPUs to train the

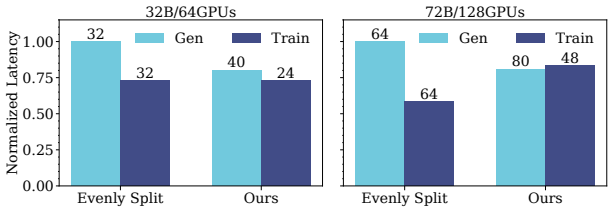

Figure 10. The iteration time breakdown compared between even resource split and our resource allocation algorithm when training 32B and 72B model on the 20K dataset.

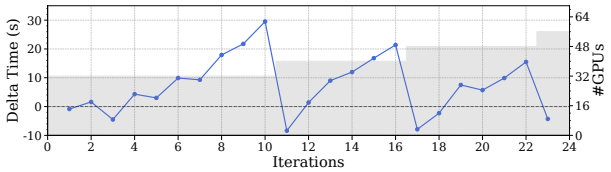

Figure 11. The delta time between the two stages when training 7B models on 32 GPUs and 10K dataset initially, then the output length is increased linearly to 20K dataset.

7B model and linearly scale the output length of the dataset to adjust the maximum output length from 10K to 20K. Figure 11 shows the delta time between the two stages in each iteration. As shown, after iteration 10 and 16, StreamRL detects the imbalance and automatically adds one node with 8 GPUs to the SGS stage to restore stage balance.

7.3 Cross-Datacenter and Heterogeneity

As discussed in §2, one promising potential of disaggregation lies in enabling each stage to utilize the most suitable hardware resources and supporting cross-datacenter training. To demonstrate this, we adopt the same settings as the end-to-end experiment with the 7B model, but move the SGS of StreamRL into a cloud-based H20 cluster with 32 GPUs. `Trainer` is still placed in the H800 cluster. We compare its performance against the original single datacenter setting. As shown in Figure 12, with heterogeneous deployment, StreamRL achieves a 1.23×–1.31× higher throughput normalized by hardware cost. This improvement comes from the higher cost-efficiency of H20 for generation workloads. Additionally, the communication overhead introduced by cross-datacenter communication is small: each iteration only requires communication during weights updates, and even for a 72B model, the transmission overhead over a 80 Gbps dedicated link is less than 10 seconds—under 2% of the total iteration time.

7.4 Algorithmic Behavior of Asynchronous RL

The effectiveness of asynchronous RL for LLMs has been observed and validated by several prior works [31, 43, 48]. To confirm this, we also conduct PPO-based RL training using Qwen2.5-32B [51] as the base model on an internal dataset. keeping all other settings the same, Figure 13 shows that the reward curves of the one-step asynchronous version closely

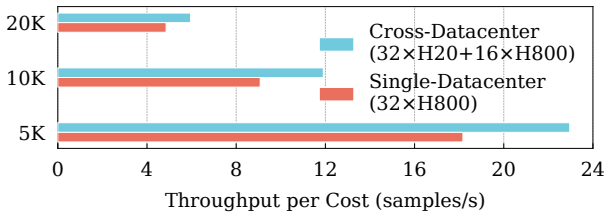

Figure 12. The throughput normalized by the hardware cost between cross- and single-datacenter deployment.

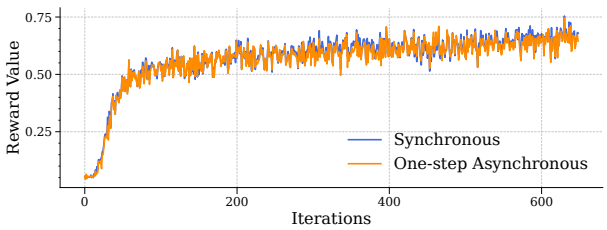

Figure 13. The reward curves between synchronous and one-step asynchronous PPO when training a 32B LLM.

match that of the synchronous version. This demonstrates that it is possible to maximize training efficiency through algorithm-system co-design without compromising model performance and convergence.

Of course, this case study only empirically verifies the feasibility of asynchronous training for specific LLM tasks; its generality and theoretical guarantees are beyond the scope of this paper. Nevertheless, even if one has concerns about the potential model convergence problem introduced by asynchrony, you can use disaggregation with streaming to improve efficiency without changing the training semantics.

8 Related Work

RL training frameworks. RL training is becoming increasingly important for LLMs to improve their performance and align their value with humans. To accelerate this process, various RL training frameworks have been proposed. One class of frameworks, such as NeMo [18] and OpenRLHF [19], partitions the GPU cluster into multiple subsets to serve different stages of RL training. They are inefficient as only one stage can be executed simultaneously, leading to resource idleness. In contrast, verl [39], RLHFuse [58], ReaL [28], and PUZZLE [25] colocate different stages on the same GPU pool to maximize resource utilization. ReaL [28] avoids underutilization by parameter reallocation. RLHFuse [58] further proposes stage fusion to reduce the idleness at the sub-stage level. PUZZLE [25] proposes lightweight context switching to reduce the switching overhead. However, they suffers from the resource coupling problem which is solved by StreamRL.

LLM inference optimizations. Many optimizations have been proposed to accelerate the LLM inference. They are also applicable to the generation phase of RL training. ORCA [52] proposes selective batching to batch requests with different lengths. vLLM [23] proposes PagedAttention to reduce memory fragmentation of various requests. FastServe [50] uses preemptive scheduling to reduce the head-of-line blocking problem of long requests. Splitwise [34] and DistServe [57] split the prefill and decoding phases to avoid interference between them. Similar to StreamRL, they also adopt the idea of resource disaggregation, but in the context of LLM inference. LoongServe [49] proposes elastic sequence parallelism to serve different requests with different degrees of parallelism. They are orthogonal to StreamRL and most of them are implemented in StreamRL to improve the generation.

LLM training optimizations. LLM training has been extensively studied in the past few years. Tensor parallelism [41], data parallelism, and pipeline parallelism [20] are widely used to parallelize the LLM training in different dimensions. Alpa [55] proposes a unified framework to automatically search for the optimal parallel strategies. CoDDL [21], Pollux [35], and ElasticFlow [16] elastically adjust the parallelism strategy to adapt to the workload. MegaScale [22] summarizes various best practices for optimizing ultra-scale training. StreamRL targets the RL training, where LLM training is just a single stage of the whole process.

9 Conclusion

In this work, we revisit the disaggregated architecture for RL training to highlight its promising advantages over the widely adopted colocation architecture: flexible resource allocation, support for heterogeneous hardware, and cross-datacenter scalability. To fully unlock the potential of disaggregation, we present StreamRL, which addresses the pipeline bubbles and skewness-induced inefficiencies present in existing disaggregated RL frameworks. Experiments show that StreamRL achieves up to a 2.66× speedup compared to the current state-of-the-art RL framework. We hope this work encourages the community to revisit disaggregation and gain a deeper understanding of its effectiveness.

References

[1] 2024. Distribution of AI training is needed. https://www.tomshardware.com/tech-industry/artificial-intelligence/microsoft-azure-cto-claims-distribution-of-ai-training-is-needed-as-ai-datacenters-approach-power-grid-limits. (2024).
[2] 2024. Introducing OpenAI o1. https://openai.com/index/openai-o3-mini/. (2024).
[3] 2024. Multi-Datacenter Training. https://semianalysis.com/2024/09/04/multi-datacenter-training-openais/. (2024).
[4] 2024. OpenAI o3-mini: Pushing the frontier of cost-effective reasoning. https://openai.com/o1/. (2024).
[5] 2024. Qwen2.5: A Party of Foundation Models! https://qwenlm.github.io/blog/qwen2.5/. (2024).
[6] 2025. Claude 3.7 Sonnet and Claude Code. https://www.anthropic.com/news/claude-3-7-sonnet. (2025).
[7] 2025. Seed-Thinking-v1.5: Advancing Superb Reasoning Models with Reinforcement Learning. https://github.com/ByteDance-Seed/Seed-Thinking-v1.5. (2025).
[8] Reza Yazdani Aminabadi, Samyam Rajbhandari, Minjia Zhang, Ammar Ahmad Awan, Cheng Li, Du Li, Elton Zheng, Jeff Rasley, Shaden Smith, Olatunji Ruwase, et al. 2022. Deepspeed inference: Enabling

efficient inference of transformer models at unprecedented scale. *arXiv* (2022).

[9] Yuntao Bai, Andy Jones, Kamal Ndousse, Amanda Askell, Anna Chen, Nova DasSarma, Dawn Drain, Stanislav Fort, Deep Ganguli, Tom Henighan, Nicholas Joseph, Saurav Kadavath, Jackson Kernion, Tom Conerly, Sheer El-Showk, Nelson Elhage, Zac Hatfield-Dodds, Danny Hernandez, Tristan Hume, Scott Johnston, Shauna Kravec, Liane Lovitt, Neel Nanda, Catherine Olsson, Dario Amodei, Tom Brown, Jack Clark, Sam McCandlish, Chris Olah, Ben Mann, and Jared Kaplan. 2022. Training a Helpful and Harmless Assistant with Reinforcement Learning from Human Feedback. *arXiv preprint arXiv:2204.05862* (2022).

[10] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. 2020. Language models are few-shot learners. *Advances in neural information processing systems* (2020).

[11] Aakanksha Chowdhery, Sharan Narang, Jacob Devlin, Maarten Bosma, Gaurav Mishra, Adam Roberts, Paul Barham, Hyung Won Chung, Charles Sutton, Sebastian Gehrmann, et al. 2022. Palm: Scaling language modeling with pathways. *arXiv preprint arXiv:2204.02311* (2022).

[12] Zihang Dai, Zhilin Yang, Yiming Yang, Jaime Carbonell, Quoc V Le, and Ruslan Salakhutdinov. 2019. Transformer-XL: Attentive language models beyond a fixed-length context. (2019).

[13] DeepSeek-AI, Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shirong Ma, Peiyi Wang, Xiao Bi, Xiaokang Zhang, Xingkai Yu, Yu Wu, Z. F. Wu, Zhibin Gou, Zhihong Shao, Zhuoshu Li, Ziyi Gao, Aixin Liu, Bing Xue, Bingxuan Wang, Bochao Wu, Bei Feng, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, Damai Dai, Deli Chen, Dongjie Ji, Erhang Li, Fangyun Lin, Fucong Dai, Fuli Luo, Guangbo Hao, Guanting Chen, Guowei Li, H. Zhang, Han Bao, Hanwei Xu, Haocheng Wang, Honghui Ding, Huajian Xin, Huazuo Gao, Hui Qu, Hui Li, Jianzhong Guo, Jiashi Li, Jiawei Wang, Jingchang Chen, Jingyang Yuan, Junjie Qiu, Junlong Li, J. L. Cai, Jiaqi Ni, Jian Liang, Jin Chen, Kai Dong, Kai Hu, Kaige Gao, Kang Guan, Kexin Huang, Kuai Yu, Lean Wang, Lecong Zhang, Liang Zhao, Litong Wang, Liyue Zhang, Lei Xu, Leyi Xia, Mingchuan Zhang, Minghua Zhang, Minghui Tang, Meng Li, Miaojun Wang, Mingming Li, Ning Tian, Panpan Huang, Peng Zhang, Qiancheng Wang, Qinyu Chen, Qiushi Du, Ruiqi Ge, Ruisong Zhang, Ruizhe Pan, Runji Wang, R. J. Chen, R. L. Jin, Ruyi Chen, Shanghao Lu, Shangyan Zhou, Shanhuang Chen, Shengfeng Ye, Shiyu Wang, Shuiping Yu, Shunfeng Zhou, Shuting Pan, S. S. Li, Shuang Zhou, Shaoqing Wu, Shengfeng Ye, Tao Yun, Tian Pei, Tianyu Sun, T. Wang, Wangding Zeng, Wanjia Zhao, Wen Liu, Wenfeng Liang, Wenjun Gao, Wenqin Yu, Wentao Zhang, W. L. Xiao, Wei An, Xiaodong Liu, Xiaohan Wang, Xiaokang Chen, Xiaotao Nie, Xin Cheng, Xin Liu, Xin Xie, Xingchao Liu, Xinyu Yang, Xinyuan Li, Xuecheng Su, Xuheng Lin, X. Q. Li, Xiangyue Jin, Xiaojin Shen, Xiaosha Chen, Xiaowen Sun, Xiaoxiang Wang, Xinnan Song, Xinyi Zhou, Xianzu Wang, Xinxia Shan, Y. K. Li, Y. Q. Wang, Y. X. Wei, Yang Zhang, Yanhong Xu, Yao Li, Yao Zhao, Yaofeng Sun, Yaohui Wang, Yi Yu, Yichao Zhang, Yifan Shi, Yiliang Xiong, Ying He, Yishi Piao, Yisong Wang, Yixuan Tan, Yiyang Ma, Yiyuan Liu, Yongqiang Guo, Yuan Ou, Yuduan Wang, Yue Gong, Yuheng Zou, Yujia He, Yunfan Xiong, Yuxiang Luo, Yuxiang You, Yuxuan Liu, Yuyang Zhou, Y. X. Zhu, Yanhong Xu, Yanping Huang, Yaohui Li, Yi Zheng, Yuchen Zhu, Yunxian Ma, Ying Tang, Yukun Zha, Yuting Yan, Z. Z. Ren, Zehui Ren, Zhangli Sha, Zhe Fu, Zhean Xu, Zhenda Xie, Zhengyan Zhang, Zhewen Hao, Zhicheng Ma, Zhigang Yan, Zhiyu Wu, Zihui Gu, Zijia Zhu, Zijun Liu, Zilin Li, Ziwei Xie, Ziyang Song, Zizheng Pan, Zhen Huang, Zhipeng Xu, Zhongyu Zhang, and Zhen Zhang. 2025. DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning. *arXiv preprint arXiv:2501.12948* (2025).

[14] Yichao Fu, Siqi Zhu, Runlong Su, Aurick Qiao, Ion Stoica, and Hao Zhang. 2024. Efficient LLM Scheduling by Learning to Rank. (2024). arXiv:cs.LG/2408.15792 https://arxiv.org/abs/2408.15792

[15] Ronald L. Graham. 1969. Bounds on multiprocessing timing anomalies. *SIAM journal on Applied Mathematics* (1969).

[16] Diandian Gu, Yihao Zhao, Yinmin Zhong, Yifan Xiong, Zhenhua Han, Peng Cheng, Fan Yang, Gang Huang, Xin Jin, and Xuanzhe Liu. 2023. Elasticflow: An elastic serverless training platform for distributed deep learning. In *ACM ASPLOS*.

[17] Arpan Gujarati, Reza Karimi, Safya Alzayat, Wei Hao, Antoine Kaufmann, Ymir Vigfusson, and Jonathan Mace. 2020. Serving DNNs like Clockwork: Performance Predictability from the Bottom Up. In *USENIX OSDI*.

[18] Eric Harper, Somshubra Majumdar, Oleksii Kuchaiev, Li Jason, Yang Zhang, Evelina Bakhturina, Vahid Noroozi, Sandeep Subramanian, Koluguri Nithin, Huang Jocelyn, Fei Jia, Jagadeesh Balam, Xuesong Yang, Micha Livne, Yi Dong, Sean Naren, and Boris Ginsburg. 2025. NeMo: a toolkit for Conversational AI and Large Language Models. (2025). https://github.com/NVIDIA/NeMo

[19] Jian Hu, Xibin Wu, Weixun Wang, Dehao Zhang, Yu Cao, et al. 2024. OpenRLHF: An Easy-to-use, Scalable and High-performance RLHF Framework. *arXiv preprint arXiv:2405.11143* (2024).

[20] Yanping Huang, Youlong Cheng, Ankur Bapna, Orhan Firat, Dehao Chen, Mia Chen, HyoukJoong Lee, Jiquan Ngiam, Quoc V Le, Yonghui Wu, et al. 2019. GPipe: Efficient Training of Giant Neural Networks using Pipeline Parallelism . In *NeurIPS*.

[21] Changho Hwang, Taehyun Kim, Sunghyun Kim, Jinwoo Shin, and KyoungSoo Park. 2021. Elastic Resource Sharing for Distributed Deep Learning. In *USENIX NSDI*.

[22] Ziheng Jiang, Haibin Lin, Yinmin Zhong, Qi Huang, Yangrui Chen, Zhi Zhang, Yanghua Peng, Xiang Li, Cong Xie, Shibiao Nong, et al. 2024. {MegaScale}: Scaling large language model training to more than 10,000 {GPUs}. In *USENIX NSDI*.

[23] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles*. 611–626.

[24] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th Symposium on Operating Systems Principles*. 611–626.

[25] Kinman Lei, Yuyang Jin, Mingshu Zhai, Kezhao Huang, Haoxing Ye, and Jidong Zhai. 2024. {PUZZLE}: Efficiently Aligning Large Language Models through {Light-Weight} Context Switch. In *2024 USENIX Annual Technical Conference (USENIX ATC 24)*. 127–140.

[26] Zhuohan Li, Lianmin Zheng, Yinmin Zhong, Vincent Liu, Ying Sheng, Xin Jin, Yanping Huang, Zhifeng Chen, Hao Zhang, Joseph E Gonzalez, et al. 2023. AlpaServe: Statistical Multiplexing with Model Parallelism for Deep Learning Serving. *arXiv* (2023).

[27] Michael Luo, Sijun Tan, Roy Huang, Ameen Patel, Alpay Ariyak, Qingyang Wu, Xiaoxiang Shi, Rachel Xin, Colin Cai, Maurice Weber, Ce Zhang, Li Erran Li, Raluca Ada Popa, and Ion Stoica. 2025. DeepCoder: A Fully Open-Source 14B Coder at O3-mini Level. https://pretty-radio-b75.notion.site/DeepCoder-A-Fully-Open-Source-14B-Coder-at-O3-mini-Level-1cf81902c14680b3bee5eb349a512a51. (2025). Notion Blog.

[28] Zhiyu Mei, Wei Fu, Kaiwei Li, Guangju Wang, Huanchen Zhang, and Yi Wu. 2024. ReaLHF: Optimized RLHF Training for Large Language Models through Parameter Reallocation. *arXiv preprint arXiv:2406.14088* (2024).

[29] Xupeng Miao, Yujie Wang, Youhe Jiang, Chunan Shi, Xiaonan Nie, Hailin Zhang, and Bin Cui. 2022. Galvatron: Efficient transformer training over multiple gpus using automatic parallelism. In *Proceedings of the VLDB Endowment*.

[30] Philipp Moritz, Robert Nishihara, Stephanie Wang, Alexey Tumanov, Richard Liaw, Eric Liang, Melih Elibol, Zongheng Yang, William Paul, Michael I. Jordan, and Ion Stoica. 2018. Ray: A Distributed Framework for Emerging AI Applications. In *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18)*.

[31] Michael Noukhovitch, Shengyi Huang, Sophie Xhonneux, Arian Hosseini, Rishabh Agarwal, and Aaron Courville. 2025. Asynchronous RLHF: Faster and More Efficient Off-Policy RL for Language Models. *International Conference on Learning Representations (ICLR)* (2025).

[32] OpenAI. 2023. GPT-4 Technical Report. (2023).

[33] Myle Ott, Sergey Edunov, Alexei Baevski, Angela Fan, Sam Gross, Nathan Ng, David Grangier, and Michael Auli. 2019. fairseq: A fast, extensible toolkit for sequence modeling. *arXiv* (2019).

[34] Pratyush Patel, Esha Choukse, Chaojie Zhang, Aashaka Shah, Íñigo Goiri, Saeed Maleki, and Ricardo Bianchini. 2024. Splitwise: Efficient generative llm inference using phase splitting. In *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*. 118–132.

[35] Aurick Qiao, Sang Keun Choe, Suhas Jayaram Subramanya, Willie Neiswanger, Qirong Ho, Hao Zhang, Gregory R. Ganger, and Eric P. Xing. 2021. Pollux: Co-adaptive Cluster Scheduling for Goodput-Optimized Deep Learning. In *USENIX OSDI*.

[36] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. 2017. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347* (2017).

[37] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, YK Li, Y Wu, et al. 2024. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300* (2024).

[38] Guangming Sheng, Chi Zhang, Zilingfeng Ye, Xibin Wu, Wang Zhang, Ru Zhang, Yanghua Peng, Haibin Lin, and Chuan Wu. 2024. Hybridflow: A flexible and efficient rlhf framework. *arXiv preprint arXiv:2409.19256* (2024).

[39] Guangming Sheng, Chi Zhang, Zilingfeng Ye, Xibin Wu, Wang Zhang, Ru Zhang, Yanghua Peng, Haibin Lin, and Chuan Wu. 2024. verl: Volcano Engine Reinforcement Learning for LLM. https://github.com/volcengine/verl. (2024).

[40] Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. 2019. Megatron-lm: Training multi-billion parameter language models using model parallelism. *arXiv preprint arXiv:1909.08053* (2019).

[41] Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. 2020. Megatron-LM: Training Multi-Billion Parameter Language Models Using Model Parallelism. (2020).

[42] Chenchen Shou, Guyue Liu, Hao Nie, Huaiyu Meng, Yu Zhou, Yimin Jiang, Wenqing Lv, Yelong Xu, Yuanwei Lu, Zhang Chen, et al. 2025. InfinitePOD: Building Datacenter-Scale High-Bandwidth Domain for LLM with Optical Circuit Switching Transceivers. *arXiv preprint arXiv:2502.03885* (2025).

[43] Kimi Team, Angang Du, Bofei Gao, Bowei Xing, Changjiu Jiang, Cheng Chen, Cheng Li, Chenjun Xiao, Chenzhuang Du, Chonghua Liao, et al. 2025. Kimi k1. 5: Scaling reinforcement learning with llms. *arXiv preprint arXiv:2501.12599* (2025).

[44] Pytorch Team. 2025. Torch Distributed RPC Framework. (2025). https://pytorch.org/docs/stable/rpc.html

[45] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, et al. 2023. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288* (2023).

[46] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. *Neural Information Processing Systems* (2017).

[47] Minjie Wang, Chien-chin Huang, and Jinyang Li. 2019. Supporting very large models using automatic dataflow graph partitioning. In *EuroSys*.

[48] Taiyi Wang, Zhihao Wu, Jianheng Liu, Jianye Hao, Jun Wang, and Kun Shao. 2025. DistRL: An Asynchronous Distributed Reinforcement Learning Framework for On-Device Control Agents. (2025). arXiv:cs.LG/2410.14803 https://arxiv.org/abs/2410.14803

[49] Bingyang Wu, Shengyu Liu, Yinmin Zhong, Peng Sun, Xuanzhe Liu, and Xin Jin. 2024. LoongServe: Efficiently Serving Long-context Large Language Models with Elastic Sequence Parallelism. *arXiv preprint arXiv:2404.09526* (2024).

[50] Bingyang Wu, Yinmin Zhong, Zili Zhang, Gang Huang, Xuanzhe Liu, and Xin Jin. 2023. Fast distributed inference serving for large language models. *arXiv preprint arXiv:2305.05920* (2023).

[51] An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chengyuan Li, Dayiheng Liu, Fei Huang, Haoran Wei, et al. 2024. Qwen2. 5 technical report. *arXiv preprint arXiv:2412.15115* (2024).

[52] Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. 2022. Orca: A Distributed Serving System for {Transformer-Based} Generative Models. In *USENIX OSDI*.

[53] Qiying Yu, Zheng Zhang, Ruofei Zhu, Yufeng Yuan, Xiaochen Zuo, Yu Yue, Tiantian Fan, Gaohong Liu, Lingjun Liu, Xin Liu, et al. 2025. DAPO: An open-source llm reinforcement learning system at scale. *arXiv preprint arXiv:2503.14476* (2025).

[54] Zili Zhang, Yinmin Zhong, Ranchen Ming, Hanpeng Hu, Jianjian Sun, Zheng Ge, Yibo Zhu, and Xin Jin. 2024. DistTrain: Addressing Model and Data Heterogeneity with Disaggregated Training for Multimodal Large Language Models. *arXiv preprint arXiv:2408.04275* (2024).

[55] Lianmin Zheng, Zhuohan Li, Hao Zhang, Yonghao Zhuang, Zhifeng Chen, Yanping Huang, Yida Wang, Yuanzhong Xu, Danyang Zhuo, Eric P. Xing, Joseph E. Gonzalez, and Ion Stoica. 2022. Alpa: Automating Inter- and Intra-Operator Parallelism for Distributed Deep Learning. In *USENIX OSDI*.

[56] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark Barrett, and Ying Sheng. 2024. SGLang: Efficient Execution of Structured Language Model Programs. (2024).

[57] Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. 2024. {DistServe}: Disaggregating prefill and decoding for goodput-optimized large language model serving. In *USENIX OSDI*.

[58] Yinmin Zhong, Zili Zhang, Bingyang Wu, Shengyu Liu, Yukun Chen, Changyi Wan, Hanpeng Hu, Lei Xia, Ranchen Ming, Yibo Zhu, et al. 2025. Rlhfuse: Efficient rlhf training for large language models with inter-and intra-stage fusion. *USENIX NSDI* (2025).

[59] Yinmin Zhong, Zili Zhang, Bingyang Wu, Shengyu Liu, Yukun Chen, Changyi Wan, Hanpeng Hu, Lei Xia, Ranchen Ming, Yibo Zhu, and Xin Jin. 2024. RLHFuse: Efficient RLHF Training for Large Language Models with Inter- and Intra-Stage Fusion. (2024). arXiv:cs.LG/2409.13221 https://arxiv.org/abs/2409.13221

[60] Ruidong Zhu, Ziheng Jiang, Chao Jin, Peng Wu, Cesar A. Stuardo, Dongyang Wang, Xinlei Zhang, Huaping Zhou, Haoran Wei, Yang Cheng, Jianzhe Xiao, Xinyi Zhang, Lingjun Liu, Haibin Lin, Li-Wen Chang, Jianxi Ye, Xiao Yu, Xuanzhe Liu, Xin Jin, and Xin Liu. 2025. MegaScale-Infer: Serving Mixture-of-Experts at Scale with Disaggregated Expert Parallelism. (2025). arXiv:cs.DC/2504.02263 https://arxiv.org/abs/2504.02263